Inhalte der Präsentation:

- Daten

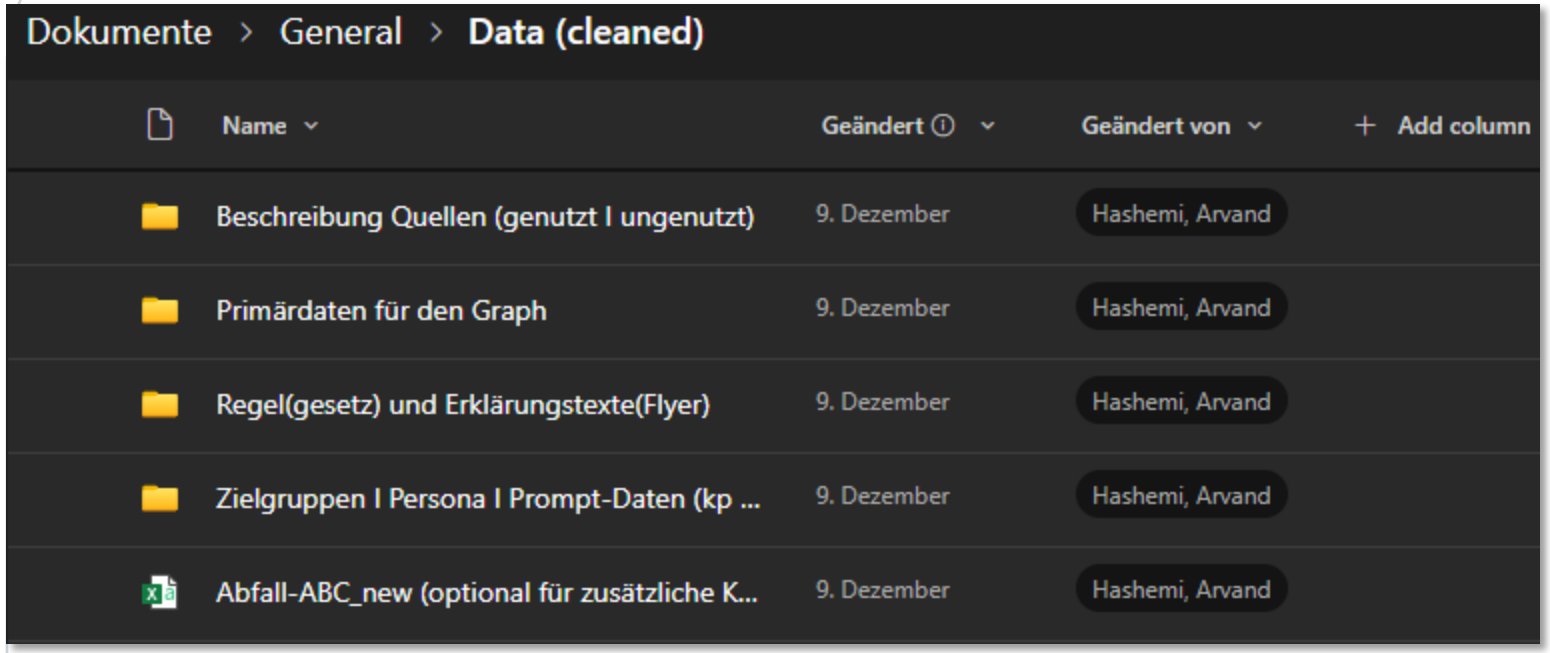
- Schema

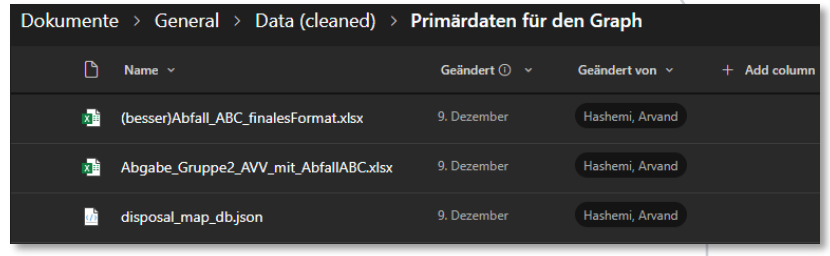
- Literatur

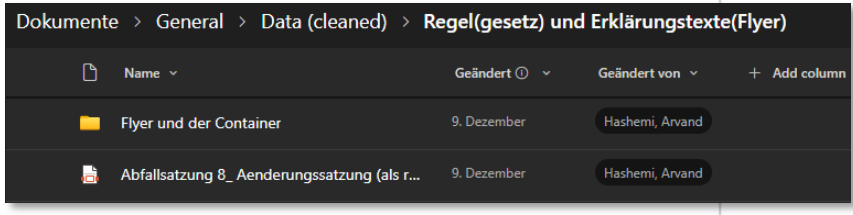
-Demo

Data Überblick

Kurz: Was steckt drin, warum ist es besser als der Rohstand und wie die Daten später im Graph genutzt werden.









Vorher vs. Nachher

Vorher (BSCW-Sammlung / Rohstand)

- 40 Dateien, ~25.67 MB Zip Ordner
- 7 Inhalts-Duplikat-Gruppen (mehrfach gleiche Dateien)
- Mehrere Varianten der gleichen Word-Flyer (Container-Dokumente)
- Zusätzliche Beifänge: Bilder / Systemdateien / Gruppen-Ordner

Nachher (Data cleaned)

- 18 Dateien, ~4.74 MB
- Keine Inhalts-Duplikate
- Eindeutige Ablage & klare Ordnerlogik
- Quellen sauber dokumentiert (genutzt / ungenutzt)

Was steckt drin? (Kernbausteine)

Die wichtigsten Artefakte – damit ihr direkt weiterarbeiten könnt.

1) Primärdaten (Graph)

- Excel: Abfall-ABC / AVV-Mapping (Finalformat)
- JSON: disposal_map_db.json (Mapping-Datenbank)
- → Grundlage für Visualisierung / Lookup / Import

2) Regeln & Erklärtexte

- PDF: Abfallsatzung (rechtliche Basis)
- PDF/Word: „was kommt wo hinein?“ + Container-Flyer

3) Optional: Zielgruppen

- Sinus-Milieus (v2) + Zuordnung
- age_groups.json + prompts.json

Was wurde bereinigt – und warum ist das besser?

Bereinigungen (konkret)

• Duplikate entfernt (inkl. mehrfach identischer PDFs/JSON/CSV)
• Mehrere Dokument-Versionen konsolidiert (Container-Flyer → 1 Stand)
• Quellenlisten ergänzt (genutzt vs. ungenutzt)

Nutzen

• Klare Strukur
• Weniger Fehler: keine Verwechslungsgefahr durch Kopien
• Besser für Aufbau Pipeline
• Nachvollziehbare Quellenbasis

Optional: Screenshot der Ordnerstruktur einfügen

Wissensgraph-Schema (Entsorgungsbot)

Wie die Daten in Neo4j als Graph strukturiert werden (für Graph-RAG & nachvollziehbare Regeln).

Schichtenmodell ...

WasteItem – Abfallart / Gegenstand …

Werkstoff (Professional-Layer) …

AVVCode …

WasteLegalClass ….

DisposalRule…

Wissensgraph-Schema (Entsorgungsbot)

Wie die Daten in Neo4j als Graph strukturiert werden (für Graph-RAG & nachvollziehbare Regeln).

Beispiele für Beziehungen ->

- (DisposalRule)-[:REQUIRES_CHECK]->(Condition).
- (Condition)-[:HAS_OPTION]->(ConditionValue).

- (DisposalRule)-[:LEADS_TO]->(WasteStream).
- (ConditionValue)-[:IMPLIES_STREAM]->(WasteStream
- (WasteStream)-[:INSTANCE_OF]->(WasteLegalClass).

- (WasteItem)-[:CLASSIFIED_AS]->(AVVCode).
- (optional) (AVVCode)-[:INSTANCE_OF]->(WasteLegalClass).
- (optional) (AVVCode)-[:IMPLIES_HAZARD]->(HazardType).

- (WasteItem)-[:HAS_RULE]->(DisposalRule).
- (WasteItem)-[:INSTANCE_OF]->(WasteLegalClass).
- (WasteItem)-[:CLASSIFIED_AS]->(AVVCode).
- (WasteItem)-[:HAS_HAZARD]->(HazardType).
- (Professional) (WasteItem)-[:MADE_OF]->(Material).

- (WasteItem)-[:MADE_OF]->(Material).
- (Material)-[:DEFAULT_DISPOSAL]->(WasteStream) – Material.
- (Material)-[:HAS_HAZARD]->(HazardType) – z. B. Bat

Das 3-Schichten-Schema

Logische Trennung für Skalierbarkeit und saubere Regeln.

1) Begriffs-Ebene (Core Entities)

Das „Was“

- Objekte & Begriffe werden definiert
- Zentral: WasteItem (z. B. „Farbeimer“)
- Rechtliche Klassifizierung über AVVCode / WasteLegalClass

2) Regel-Ebene (Logic & Rules)

Das „Wie“

- Herzstück: DisposalRule verknüpft Item → Entsorgungsweg
- Bedingungen (Condition): „Ist es flüssig?“
- Anweisungen (Instruction): „Nicht in den Ausguss“

3) Infrastruktur-Ebene (Infrastructure)

Das „Wo“

- Physische Verortung: Container, Räume, Gebäude, Facilities
- WasteStreams enden in konkreten Behältern
- Campus-Hierarchie: Building → Room → Container

Schema – Ergänzungen

Core (MVP) vs. Professional-Layer + Regeln, die oft vergessen werden.

Core

- WasteItem, AVVCode, WasteLegalClass
- DisposalRule + Condition/ConditionValue
- WasteStream + (Container / Room / Facility)
- LegalProvision (für Begründung/Compliance)

Professional-Layer (iterativ ausbauen)

- Material + MADE_OF (Circular-Economy / Materiallogik)
- HazardType (gefährlich, batteriehaltig, …)
- Regel-Priorität: scope + OVERRIDES (Campus > Stadt)
- Validierung: ALLOWS_AVV (Stream darf AVV-Code aufnehmen)

Mini-Mapping: Daten → Graph (Beispiele)

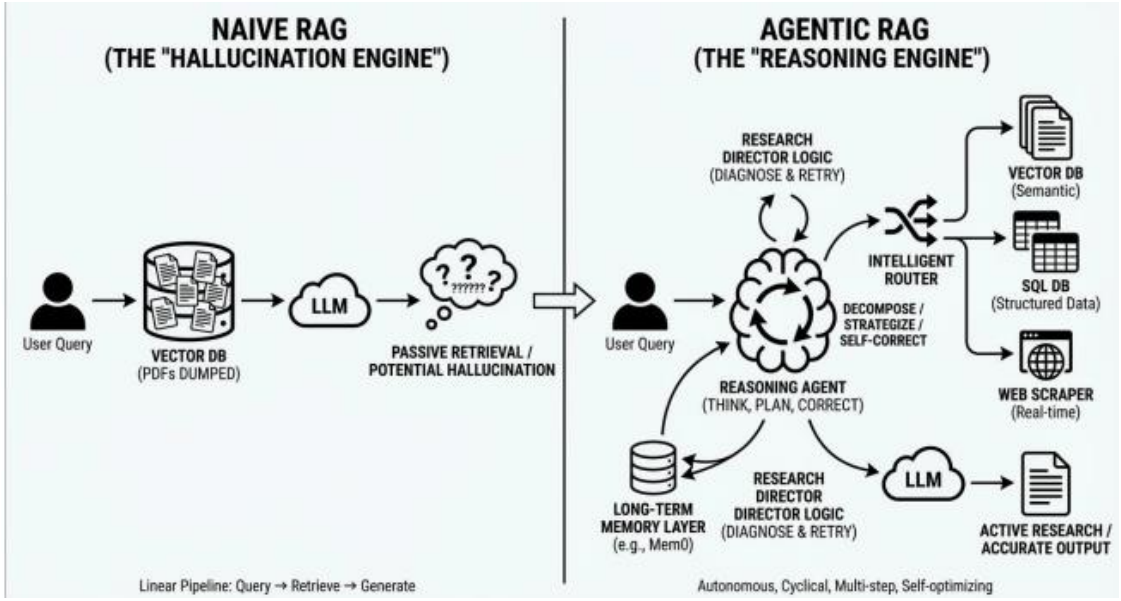
- Abfall-ABC „Abfallart“ → WasteItem.name | „Synonyme“ → WasteItem.synonyms
- Abfall-ABC „Entsorgungsweg“ → WasteStream (über DisposalRule.LEADS_TO)
- Abfall-ABC „Hinweise“ → Conditions (Wenn/Falls…) & Instructions (Imperative)
- AVV „AVV_Schlüssel“ → AVVCode.code | Sternchen → hazardous=true

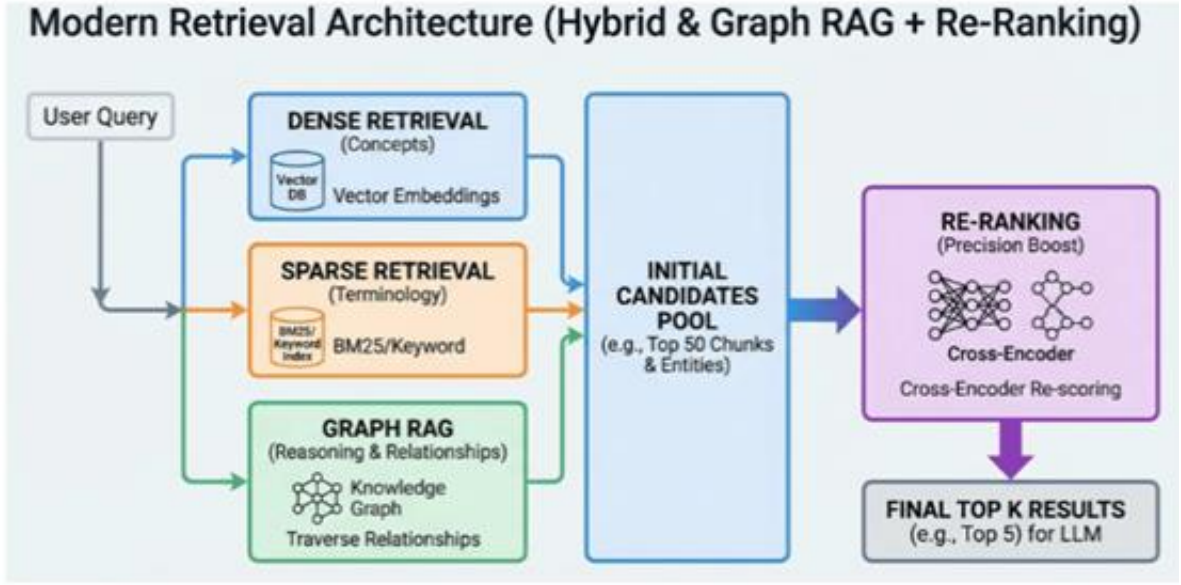
Literatur

Agentic RAG: Was ist das und warum?

- Agentic RAG = RAG mit einem Agenten, der plant, Tools nutzt und iterativ recherchiert.
- Unterschied zu klassischem RAG: nicht „einmal suchen → antworten“, sondern mehrere Zyklen (plan → retrieve → check → refine).
- Routing nach Intent: Vektor-Suche für Semantik, Keyword/BM25 für exakte Begriffe, Graph/DB für Beziehungen und Fakten.
- Self-Correction: fehlende oder widersprüchliche Belege triggern Query-Rewrite, neue Quellen, oder alternative Strategien.
- Outcome: weniger Halluzinationen, bessere Nachvollziehbarkeit (Citations/Belege), höhere Robustheit in Produktion.

Agentic Workflow: Hybrid + Graph RAG + Re-Ranking





Takeaway: Agent baut erst einen breiten Kandidaten-Pool (dense + sparse + graph) und verdichtet dann via Re-Ranking zu Top-K Belegen für die Antwort.